

**LAPORAN PRAKTIKUM
KECERDASAN BUATAN**

K-Nearest Neighbors



Fani Namesia R. Sianturi

42324045

D3 Teknologi Informasi

**INSTITUT TEKNOLOGI DEL
FAKULTAS VOKASI
TAHUN AJARAN
2026/2027**

Dalam praktikum ini, algoritma K-Nearest Neighbors (KNN) digunakan untuk melakukan klasifikasi dengan dua dataset berbeda. Dataset pertama adalah Iris, yang diambil langsung dari Scikit-Learn. Dataset ini sering digunakan dalam karena memiliki empat fitur dan tiga kelas spesies bunga, sehingga cocok untuk menguji performa algoritma klasifikasi. Data dikonversi ke dalam DataFrame Pandas agar lebih mudah diolah dan divisualisasikan. Selain itu, terdapat dataset sederhana lainnya yang digunakan untuk memahami cara kerja KNN dalam situasi berbeda.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from pandas.plotting import parallel_coordinates
# Load dataset Iris dari scikit-learn
iris = load_iris()
```

✓ 15.3s

```
# Konversi data menjadi DataFrame Pandas
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["Species"] = iris.target_names[iris.target]
# Definisikan X (fitur) dan y (target/label)
X = df.drop(columns=["Species"])
y = df["Species"]
print(X)
```

[2] ✓ 0.0s

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
...	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

[150 rows x 4 columns]

y

[3] ✓ 0.0s

0	setosa
1	setosa
2	setosa
3	setosa
4	setosa
...	...
145	virginica
146	virginica
147	virginica
148	virginica
149	virginica

Name: Species, Length: 150, dtype: str

Featut Space Dataset Iris memiliki empat fitur, yaitu sepal length, sepal width, petal length, dan petal width. Setiap fitur dihitung sebagai satu dimensi, sehingga feature space Iris berdimensi empat. Kita hanya bisa menggambar data sampai tiga sumbu (2D atau 3D), jadi data empat dimensi tidak bisa divisualisasikan dengan scatter plot biasa.

Untuk mengatasinya digunakan Parallel Coordinates Plot. Sumbu-sumbunya tidak dibuat saling tegak lurus, tetapi dijabarkan berdampingan sebagai garis vertikal, satu untuk setiap fitur. Setiap bunga kemudian digambar sebagai satu garis yang menghubungkan nilainya pada tiap sumbu, dan warna garis menunjukkan spesiesnya.

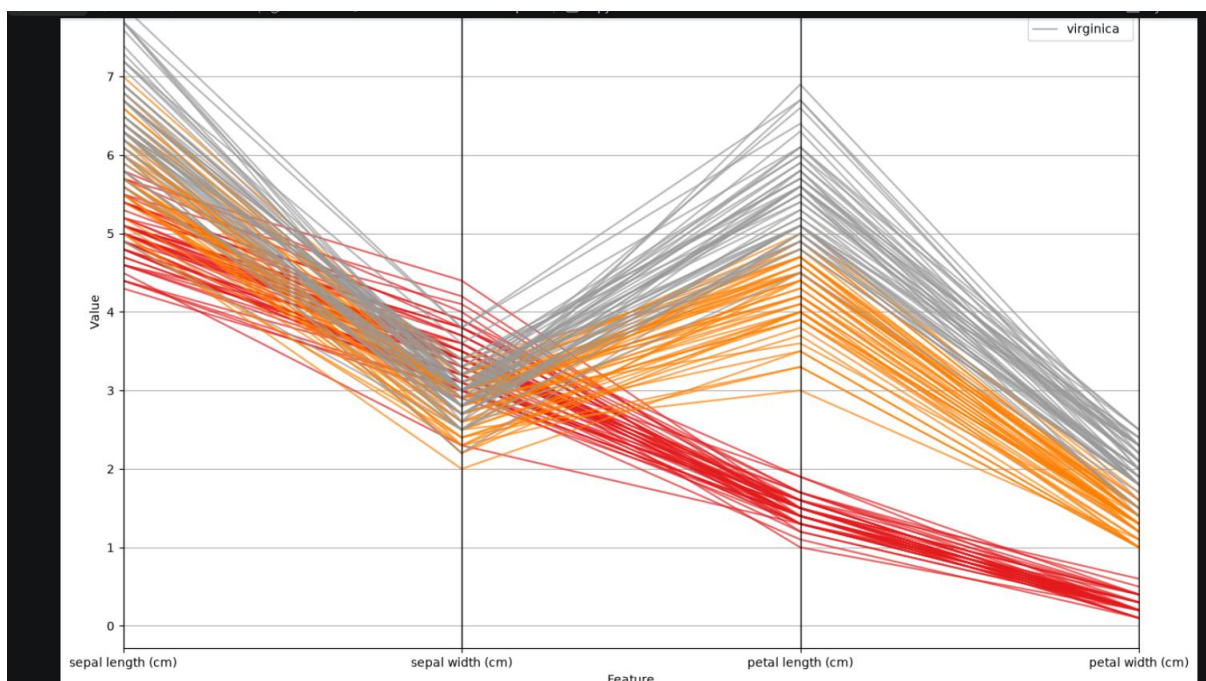
Cara membaca plot ini:

- Garis dengan warna yang sama dan mengelompok berarti kelas tersebut mudah dibedakan.
- Setosa terpisah jelas dari yang lain, terutama pada sumbu petal.
- Versicolor dan virginica agak menumpuk, tetapi mulai terpisah pada sumbu petal.

Dari sini terlihat bahwa fitur petal paling berguna untuk membedakan spesies, sehingga KNN diperkirakan bekerja dengan baik pada dataset ini.

```
# Plot Parallel Coordinates
plt.figure(figsize=(15, 10))
parallel_coordinates(df, class_column="Species", colormap=plt.cm.Set1, alpha=0.7)
plt.title("Parallel Coordinates Plot for Iris Dataset")
plt.xlabel("Feature")
plt.ylabel("Value")
plt.grid(True)
plt.show()
```

[4] ✓ 0.7s



Kode ini digunakan untuk mengubah label kelas yang berupa teks (setosa, versicolor, virginica) menjadi angka, karena algoritma machine learning bekerja dengan data numerik, bukan teks. Untuk itu digunakan LabelEncoder dari Scikit-Learn. Pada kode tersebut, LabelEncoder() membuat objek encoder, lalu fit_transform(y) melakukan dua hal sekaligus, yaitu mempelajari kelas unik yang ada (fit) dan mengubahnya menjadi angka (transform).

Angka diberikan berdasarkan urutan alfabet, sehingga setosa menjadi 0, versicolor menjadi 1, dan virginica menjadi 2. Hasilnya berupa array yang berisi 50 angka 0, 50 angka 1, dan 50 angka 2, sesuai jumlah sampel tiap spesies. Angka-angka ini hanya berfungsi sebagai kode pengganti nama kelas, bukan tingkatan atau nilai, sehingga virginica (2) tidak berarti lebih besar dari setosa (0). Objek lab disimpan karena nantinya dapat digunakan untuk mengubah hasil prediksi kembali menjadi nama spesies dengan lab.inverse_transform().

```
from sklearn.preprocessing import LabelEncoder
lab = LabelEncoder()
y = lab.fit_transform(y)
y
```

[5] ✓ 0.1s

```
... array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
          0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
          0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
          1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
          1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
          2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
          2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
          2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

```
from sklearn.model_selection import train_test_split
X_train, X_test = train_test_split(X, test_size=0.2, random_state=0)
y_train, y_test = train_test_split(y, test_size=0.2, random_state=0)
```

[6] ✓ 0.0s

- Mengimpor KNeighborsClassifier dari Scikit-Learn untuk melakukan klasifikasi menggunakan algoritma K-Nearest Neighbors (KNN).
- Mengimpor accuracy_score, yang nantinya digunakan untuk mengukur akurasi model setelah pelatihan.
- Menentukan jarak dengan menggunakan 'euclidean'. Alternatif lain bisa menggunakan metric='minkowski'

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
# k = 3
knn = KNeighborsClassifier(n_neighbors=3, metric='euclidean')
knn.fit(X_train, y_train)
```

[7] ✓ 0.7s

...

▼ KNeighborsClassifier ⓘ ?

- Parameters
- Fitted attributes

```
y_pred = knn.predict(X_test)
y_pred
```

[8] ✓ 0.0s

...

```
array([2, 1, 0, 2, 0, 2, 0, 1, 1, 1, 2, 1, 1, 1, 2, 0, 1, 1, 0, 0, 2, 1,
       0, 0, 2, 0, 0, 1, 1, 0])
```

```
result = accuracy_score(y_test, y_pred)
result
```

[9] ✓ 0.0s

...

```
0.9666666666666667
```

Klasifikasi dengan algoritma KNN menggunakan dataset athletes.csv Dataset ini digunakan untuk menentukan apakah seorang atlet akan masuk draft (Yes/No) berdasarkan dua fitur utama:

- speed → Kecepatan atlet
- agility → Kelincahan atlet

Kolom draft adalah target (label kelas) yang menunjukkan apakah atlet dipilih dalam draft atau tidak.

```
df = pd.read_csv('athletes.csv')
print(df)
```

[11] ✓ 0.0s

...		id	speed	agility	draft
	0	1	2.50	6.00	no
	1	2	3.75	8.00	no
	2	3	2.25	5.50	no
	3	4	3.25	8.25	no
	4	5	2.75	7.50	no
	5	6	4.50	5.00	no
	6	7	3.50	5.25	no
	7	8	3.00	3.25	no
	8	9	4.00	4.00	no
	9	10	4.25	3.75	no
	10	11	2.00	2.00	no
	11	12	5.00	2.50	no
	12	13	8.25	8.50	no
	13	14	5.75	8.75	yes
	14	15	4.75	6.25	yes
	15	16	5.50	6.75	yes
	16	17	5.25	9.50	yes
	17	18	7.00	4.25	yes
	18	19	7.50	8.00	yes
	19	20	7.25	5.75	yes

```
feature_columns = ['speed', 'agility']
x = df[feature_columns].values
y = df['draft'].values
```

[12] ✓ 0.0s

▷ ▾

```
x
```

[13] ✓ 0.0s

```
array([[2.5 , 6.  ],
       [3.75, 8.  ],
       [2.25, 5.5 ],
       [3.25, 8.25],
       [2.75, 7.5 ],
       [4.5 , 5.  ],
       [3.5 , 5.25],
       [3. , 3.25],
       [4. , 4.  ],
       [4.25, 3.75],
       [2. , 2.  ],
       [5. , 2.5 ],
       [8.25, 8.5 ],
       [5.75, 8.75],
       [4.75, 6.25],
       [5.5 , 6.75],
       [5.25, 9.5 ],
       [7. , 4.25],
       [7.5 , 8.  ],
       [7.25, 5.75]])
```

```
y
[14] ✓ 0.0s

... <StringArray>
[ 'no', 'no', 'no', 'no', 'no', 'no', 'no', 'no', 'no', 'no', 'no', 'no',
  'no', 'no', 'yes', 'yes', 'yes', 'yes', 'yes', 'yes', 'yes']
Length: 20, dtype: str
```

```
#encode data y menjadi numerik
en = LabelEncoder()
y = en.fit_transform(y)
y
[15] ✓ 0.0s

... array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1])
```

```
from numpy import where
df['key1'] = (y)
colors = np.where(df["key1"]==0, 'r', 'g')
print(colors)
df.plot.scatter(x=1,y=2,c=colors)
plt.show()
[16] ✓ 0.1s

... ['r' 'r' 'r' 'r' 'r' 'r' 'r' 'r' 'r' 'r' 'r' 'r' 'r' 'g' 'g' 'g' 'g' 'g'
  'g' 'g']
```

```
# split train test
X_train, X_test = train_test_split(X, test_size=0.2, random_state=0)
y_train, y_test = train_test_split(y, test_size=0.2, random_state=0)
[17] ✓ 0.0s
```

```
knn = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
knn.fit(X_train, y_train)
[18] ✓ 0.0s
```

```
... KNeighborsClassifier ⓘ ⓘ
  Parameters
  Fitted attributes
```

```
y_pred = knn.predict(X_test)
y_pred
[19] ✓ 0.0s

... array([1, 1, 1, 0])
```

```
result = accuracy_score(y_test, y_pred)
result
[20] ✓ 0.0s

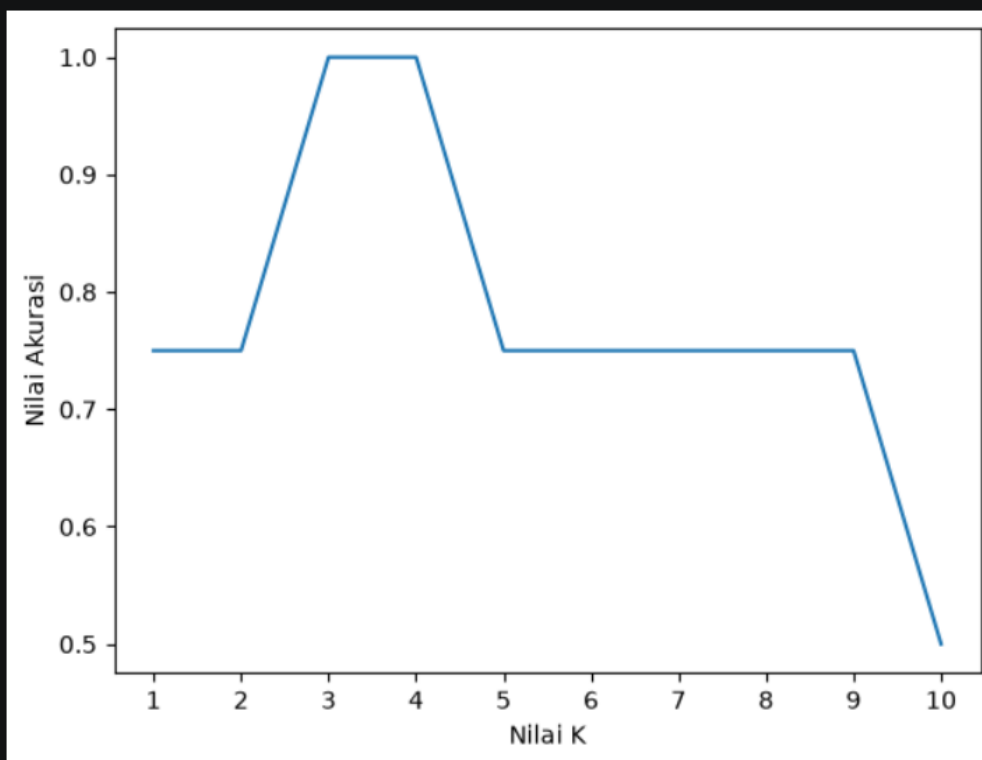
... 0.75
```



```
array_k = []
for i in range(1,11):
    knn = KNeighborsClassifier(n_neighbors=i, metric='euclidean')
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)
    # Calculate accuracy
    result = accuracy_score(y_test, y_pred)
    # Store the accuracy result
    array_k.append(result)
plt.plot(array_k)
plt.ylabel('Nilai Akurasi')
KNeighborsClassifier(metric='euclidean')
plt.xlabel('Nilai K')
plt.xticks(np.arange(10),('1','2','3','4','5','6','7','8','9','10'))
plt.show()
```

[21] ✓ 0.3s

...



Tugas

Prediksi apakah atlet dengan data berikut masuk draft yes/no

1. input = [7.5, 7.0]

2. input = [4.0, 6.5]

Untuk dataset iris, prediksi input data berikut ini masuk ke species apa:

1. input_data = [[7.1, 1.2, 5.0, 2.0]]

2. input_data = [[5.1, 3.5, 1.4, 0.2]]

3. input_data = [[6.4, 3.7, 4.2, 2.2]]

```
Tugas Praktikum W04S02
Nama: Fani Namesia R. Sianturi
Nim: 42324045
```

```
Prediksi apakah atlet dengan data berikut masuk draft yes/no
```

```
1. input = [7.5, 7.0]
```

```
2. input = [4.0, 6.5]
```

```
Untuk dataset iris, prediksi input data berikut ini masuk ke species apa:
```

```
1. input_data = [[7.1, 1.2, 5.0, 2.0]]
```

```
2. input_data = [[5.1, 3.5, 1.4, 0.2]]
```

```
3. input_data = [[6.4, 3.7, 4.2, 2.2]]
```

```
1. # Athletes (jalankan setelah knn dilatih dengan n_neighbors=3 pada data athletes)
knn = KNeighborsClassifier(n_neighbors=3, metric='euclidean')
knn.fit(X_train, y_train)
pred = knn.predict([[7.5, 7.0], [4.0, 6.5]])
print(en.inverse_transform(pred))
```

```
✓ 0.0s
```

```
['yes' 'yes']
```

```

#2
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
import pandas as pd

iris = load_iris()
X_iris = pd.DataFrame(iris.data, columns=iris.feature_names)
y_iris = iris.target

Xi_train, Xi_test = train_test_split(X_iris, test_size=0.2, random_state=0)
yi_train, yi_test = train_test_split(y_iris, test_size=0.2, random_state=0)

knn_iris = KNeighborsClassifier(n_neighbors=3, metric='euclidean')
knn_iris.fit(Xi_train, yi_train)

input_data = pd.DataFrame(
    [[7.1, 1.2, 5.0, 2.0],
     [5.1, 3.5, 1.4, 0.2],
     [6.4, 3.7, 4.2, 2.2]],
    columns=iris.feature_names)

pred = knn_iris.predict(input_data)
print(iris.target_names[pred])

```

[25] ✓ 0.0s

... ['virginica' 'setosa' 'versicolor']